Training a Causal Language Model from Scratch

Introduction

So far, we've mostly taken pretrained models and fine-tuned them for new tasks — reusing the weights from pretraining. As seen in Chapter 1, this approach is called **Transfer Learning**, and it's the most successful strategy for applying Transformer models in practice, especially when labeled data is scarce.

In this chapter, we'll take a different path. We'll build a **completely new model from scratch**.

When Should You Train from Scratch?

This makes sense when:

- You have **a large amount of data**
- That data is **completely different** from the pretraining data of existing models

However, pretraining a language model requires **far more** compute resources than fine-tuning.

Some cases where building a new model is justified:

- **Musical notes** (music notation)
- **Molecular sequences** (e.g., DNA sequences)
- **Programming languages** (source code)

For programming languages, this has become especially popular recently. Tools like **TabNine** and **GitHub Copilot** (powered by OpenAI's Codex model) can generate long code sequences. For this kind of **text generation** task, the most suitable architecture is an **auto-regressive** or **causal language model** — like GPT-2.

What Are We Building in This Chapter?

We'll build a **scaled-down code generation model**. Instead of writing full functions or classes, we'll focus on **single-line code completion**.

Our main focus will be Python's **data science stack**:

- `matplotlib`
- `seaborn`
- `pandas`
- `scikit-learn`

When working with these libraries, you often need to look up specific commands — if a model could autocomplete those calls for you, that would be incredibly useful!

In Chapter 6, we built an efficient tokenizer for processing Python source code. Now we'll apply that tokenizer to a Python code corpus gathered from GitHub repositories and train a model using the **Trainer API** and **Accelerate**.

Step 1 — Gathering the Data

The Codeparrot Dataset

Python code is readily available from repositories like GitHub. For the Transformers textbook, every Python repository was scraped to build a dataset for pretraining a large GPT-2 model. This GitHub dump — roughly **180 GB** containing **20 million Python files** — is called **codeparrot** and has been shared on the Hugging Face Hub.

However, training on the entire corpus would take enormous time and compute. We only need the portion related to the Python data science stack. So we'll filter the codeparrot dataset to keep only files that contain at least one of our target libraries.

Instead of downloading the entire dataset, we'll use the **streaming** feature to filter on the fly.

Keyword Filter Function

```
def any_keyword_in_string(string, keywords):
    for keyword in keywords:
        if keyword in string:
            return True
    return False
```

Let's test it on two examples:

```
filters = ["pandas", "sklearn", "matplotlib", "seaborn"]
example_1 = "import numpy as np"
example_2 = "import pandas as pd"

print(
```

```
    any_keyword_in_string(example_1, filters), any_keyword_in_string(example_2, filters)
)
```

Output:

```
False True
```

Working correctly. Now let's build a function that uses this to filter a streaming dataset:

```
from collections import defaultdict
from tqdm import tqdm
from datasets import Dataset


def filter_streaming_dataset(dataset, filters):
    filtered_dict = defaultdict(list)
    total = 0
    for sample in tqdm(iter(dataset)):
        total += 1
        if any_keyword_in_string(sample["content"], filters):
            for k, v in sample.items():
                filtered_dict[k].append(v)
    print(f"{len(filtered_dict['content'])/total:.2%} of data after filtering.")
    return Dataset.from_dict(filtered_dict)
```

Now let's apply this function to the streaming dataset:

```
# This cell will take a very long time to execute, so you should skip it and go to
# the next one!
from datasets import load_dataset

split = "train"  # "valid"
filters = ["pandas", "sklearn", "matplotlib", "seaborn"]

data = load_dataset(f"transformersbook/codeparrot-{split}", split=split, streaming=True)
filtered_data = filter_streaming_dataset(data, filters)
```

Output:

```
3.26% of data after filtering.
```

Even though only 3% of the original dataset remains, it's still substantial — **6 GB** and **600,000 Python scripts**!

> Filtering the entire dataset can take **2–3 hours** depending on your machine and bandwidth. To skip this step, we can directly download the pre-filtered dataset from the Hub:

```
from datasets import load_dataset, DatasetDict

ds_train = load_dataset("huggingface-course/codeparrot-ds-train", split="train")
ds_valid = load_dataset("huggingface-course/codeparrot-ds-valid", split="validation")

raw_datasets = DatasetDict(
    {
        "train": ds_train,  # .shuffle().select(range(50000)),
        "valid": ds_valid,  # .shuffle().select(range(500))
    }
)

raw_datasets
```

Output:

```
DatasetDict({
    train: Dataset({
        features: ['repo_name', 'path', 'copies', 'size', 'content', 'license'],
        num_rows: 606720
    })
    valid: Dataset({
        features: ['repo_name', 'path', 'copies', 'size', 'content', 'license'],
        num_rows: 3322
    })
})
```

> **Tip:** Language model pretraining takes a long time. First, uncomment the two partial lines above to run the training loop on a small subset and verify that everything works correctly and that the model is being saved properly. There's nothing more painful than a failure at the very last step of a training run!

Inspecting the Dataset

Let's look at the first 200 characters of each field:

```
for key in raw_datasets["train"][0]:
    print(f"{key.upper()}: {raw_datasets['train'][0][key][:200]}")
```

Output:

```
'REPO_NAME: kmike/scikit-learn'
'PATH: sklearn/utils/__init__.py'
'COPIES: 3'
'SIZE: 10094'
"'CONTENT: """
The :mod:`sklearn.utils` module includes various utilites.
"""

from collections import Sequence

import numpy as np
from scipy.sparse import issparse
import warnings

from .murmurhash import murm
LICENSE: bsd-3-clause"'
```

The `content` field contains the code our model will be trained on.

Step 2 — Preprocessing the Dataset

Choosing a Context Size

The first task is tokenizing the data. Since our goal is mainly to autocomplete short function calls, we can keep the context size relatively **small**. This means:

- The model trains **much faster**
- It requires **less memory**

If your application needs more context (e.g., reading an entire file including function definitions to write unit tests), you'd increase this number — but the GPU memory footprint would grow accordingly.

We'll set the context size to **128 tokens** (GPT-2 and GPT-3 use 1,024 and 2,048, respectively).

Chunking Strategy

Most documents will contain more than 128 tokens. Simply truncating would throw away a large portion of the dataset.

Instead, we'll use the `return_overflowing_tokens` option to tokenize the full input and split it into **multiple chunks** (as in Chapter 6). We'll also use `return_length` to automatically get the length of each chunk. The final chunk of a document is often shorter — we'll discard those, since we have enough data that padding won't be an issue.

Let's examine the first two examples:

```
from transformers import AutoTokenizer

context_length = 128
tokenizer = AutoTokenizer.from_pretrained("huggingface-course/code-search-net-tokenizer")

outputs = tokenizer(
    raw_datasets["train"][:2]["content"],
    truncation=True,
    max_length=context_length,
    return_overflowing_tokens=True,
    return_length=True,
)

print(f"Input IDs length: {len(outputs['input_ids'])}")
print(f"Input chunk lengths: {(outputs['length'])}")
print(f"Chunk mapping: {outputs['overflow_to_sample_mapping']}")
```

Output:

```
Input IDs length: 34
Input chunk lengths: [128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128,
128, 128, 128, 128, 128, 117, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128,
41]
```

Chunk mapping: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Two examples produced **34 segments** in total. Looking at the chunk lengths, the final chunks of the two documents have 117 and 41 tokens respectively — a small fraction of all chunks, so discarding them is perfectly fine. The `overflow_to_sample_mapping` field tells us which chunk came from which input sample.

Tokenize Function

```
def tokenize(element):
    outputs = tokenizer(
        element["content"],
        truncation=True,
        max_length=context_length,
        return_overflowing_tokens=True,
        return_length=True,
    )
    input_batch = []
    for length, input_ids in zip(outputs["length"], outputs["input_ids"]):
        if length == context_length:
            input_batch.append(input_ids)
    return {"input_ids": input_batch}


tokenized_datasets = raw_datasets.map(
    tokenize, batched=True, remove_columns=raw_datasets["train"].column_names
)
tokenized_datasets
```

Output:

```
DatasetDict({
    train: Dataset({
        features: ['input_ids'],
        num_rows: 16702061
    })
    valid: Dataset({
        features: ['input_ids'],
        num_rows: 93164
    })
```

})

We now have **16.7 million examples of 128 tokens** — roughly **2.1 billion tokens** in total.

For reference: OpenAI's GPT-3 and Codex were trained on **300 billion and 100 billion tokens** respectively. Our goal isn't to compete with those models — we simply want to build a fast **autocomplete tool** for data scientists.

> **Try it out!** With a small context window, discarding short chunks isn't a major concern. But if you increase the context size or work with a corpus of short documents, the proportion of discarded chunks will grow. A more efficient approach is to concatenate all tokenized samples in a batch using the `eos_token_id` as a separator and then chunk the result. As an exercise, modify the `tokenize()` function to do this. Remember to set `truncation=False`.

Step 3 — Initializing a New Model

The dataset is ready — time to build the model!

We'll use the same configuration as **GPT-2 small**. We load the pretrained configuration, adjust the vocabulary size to match our tokenizer, and pass in the `bos_token_id` and `eos_token_id`:

```
from transformers import AutoTokenizer, GPT2LMHeadModel, AutoConfig

config = AutoConfig.from_pretrained(
    "gpt2",
    vocab_size=len(tokenizer),
    n_ctx=context_length,
    bos_token_id=tokenizer.bos_token_id,
    eos_token_id=tokenizer.eos_token_id,
)
```

Now let's create a new model from this configuration. Notice that for the first time we're not calling `from_pretrained()` — because we're initializing a model ourselves:

```
model = GPT2LMHeadModel(config)
model_size = sum(t.numel() for t in model.parameters())
print(f"GPT-2 size: {model_size/1000**2:.1f}M parameters")
```

Output:

GPT-2 size: 124.2M parameters

Our model has **124 million parameters** to tune.

Step 4 — Creating the Data Collator

Before starting training, we need to create a **data collator** that handles batch construction.

We'll use `DataCollatorForLanguageModeling` — designed specifically for language modeling. It doesn't just stack and pad batches; it also creates **language model labels**. In causal language modeling, the input itself serves as the label (simply shifted by one step). This collator creates labels on the fly during training, so there's no need to duplicate the `input_ids`.

`DataCollatorForLanguageModeling` supports both masked language modeling (MLM) and causal language modeling (CLM). The default is MLM, but setting `mlm=False` switches to CLM:

```
from transformers import DataCollatorForLanguageModeling

tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)
```

Let's look at an example:

```
out = data_collator([tokenized_datasets["train"][i] for i in range(5)])
for key in out:
    print(f"{key} shape: {out[key].shape}")
```

Output:

```
input_ids shape: torch.Size([5, 128])
attention_mask shape: torch.Size([5, 128])
labels shape: torch.Size([5, 128])
```

The examples are stacked and all tensors have the same shape.

> **Important:** The shifting needed to align inputs and labels happens inside the model itself, so the data collator simply creates the labels as a copy of the inputs.

Step 5 — Training the Model with Trainer

Logging into the Hub

from huggingface_hub import notebook_login

notebook_login()

In the terminal:

huggingface-cli login

Defining Training Arguments

We'll use a **cosine learning rate schedule** with some warmup, and an effective batch size of **256** (`per_device_train_batch_size × gradient_accumulation_steps`).

Gradient accumulation is used when a full batch doesn't fit in memory — it gradually builds up the gradient over several forward/backward passes.

from transformers import Trainer, TrainingArguments

```
args = TrainingArguments(
    output_dir="codeparrot-ds",
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    evaluation_strategy="steps",
    eval_steps=5_000,
    logging_steps=5_000,
    gradient_accumulation_steps=8,
    num_train_epochs=1,
    weight_decay=0.1,
    warmup_steps=1_000,
    lr_scheduler_type="cosine",
    learning_rate=5e-4,
    save_steps=5_000,
    fp16=True,
    push_to_hub=True,
)

trainer = Trainer(
```

```
    model=model,
    tokenizer=tokenizer,
    args=args,
    data_collator=data_collator,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["valid"],
)
```

Starting Training

```
trainer.train()
```

Training will take time — roughly **20 hours** for the full training set, or **2 hours** for the subset. Be patient!

Pushing to the Hub

```
trainer.push_to_hub()
```

> **Try it out!** In just ~30 lines of code plus `TrainingArguments`, we went from raw text all the way to training GPT-2. Try it with your own dataset!
>
> **Tip:** If you have access to a machine with multiple GPUs, run the training there. Trainer automatically manages multiple devices, making training significantly faster.

Step 6 — Testing Code Generation with a Pipeline

Now for the real test — let's see how well the model actually performs!

First, let's wrap the model in a **text generation pipeline** and move it to the GPU for faster generation (if available):

```
import torch
from transformers import pipeline

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
pipe = pipeline(
    "text-generation", model="huggingface-course/codeparrot-ds", device=device
)
```

Test 1 — Creating a Scatter Plot

```
txt = """\
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create scatter plot with x, y
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

Output:

```
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create scatter plot with x, y
plt.scatter(x, y)

# create scatter
```

Spot on! `plt.scatter(x, y)` — exactly what's needed.

Test 2 — Creating a Pandas DataFrame

```
txt = """\
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create dataframe from x and y
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

Output:

```
# create some data
x = np.random.randn(100)
y = np.random.randn(100)
```

```
# create dataframe from x and y
df = pd.DataFrame({'x': x, 'y': y})
df.insert(0,'x', x)
for
```

The right answer is there — `pd.DataFrame({'x': x, 'y': y})`. Afterwards it redundantly inserts the `x` column again and the `for` loop is left incomplete because of the token limit.

Test 3 — GroupBy Operation

```
txt = """\
# dataframe with profession, income and name
df = pd.DataFrame({'profession': x, 'income':y, 'name': z})

# calculate the mean income per profession
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

Output:

```
# dataframe with profession, income and name
df = pd.DataFrame({'profession': x, 'income':y, 'name': z})

# calculate the mean income per profession
profession = df.groupby(['profession']).mean()

# compute the
```

Excellent! `df.groupby(['profession']).mean()` — exactly the right approach.

Test 4 — Scikit-learn Random Forest

```
txt = """
# import random forest regressor from scikit-learn
from sklearn.ensemble import RandomForestRegressor

# fit random forest model with 300 estimators on X, y:
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

Output:

```
# import random forest regressor from scikit-learn
from sklearn.ensemble import RandomForestRegressor

# fit random forest model with 300 estimators on X, y:
rf = RandomForestRegressor(n_estimators=300, random_state=random_state, max_depth=3)
rf.fit(X, y)
rf
```

Impressive! It correctly initializes `RandomForestRegressor` and calls `fit`.

Evaluating the Results

From these few examples, the model has clearly learned a solid amount of Python data science syntax. Of course, a more thorough evaluation would be needed before any real-world deployment.

Sometimes, training a model for a specific use case requires additional customization — such as dynamically updating the batch size or skipping bad examples with conditional training logic. For that kind of work, subclassing `Trainer` or writing a training loop from scratch may be necessary — and that's exactly where **Accelerate** comes in.